Session 1.5

The Workbench: Python, VS Code, Git and GitHub

From a clean laptop to Python code that runs again, on any machine

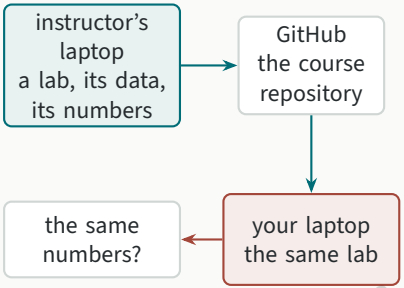


Redistribution prohibited without permission

Zhigang Feng

© Zhigang Feng

Why a Workbench



the workbench decides whether the last arrow holds

- Every later session hands you a lab notebook that runs offline on a laptop CPU: no API key, no GPU.
- Whether it runs on *your* laptop depends on what you set up once: the course files, a Python, its packages, an editor, and a record of what you changed.
- This deck builds that, teaches the Python the labs are written in, and ends with the test: the Session 1 lab.
- **A result counts only if it runs again: tomorrow, on another machine, by someone else.**

© Zhigang Feng
Redistribution prohibited without permission

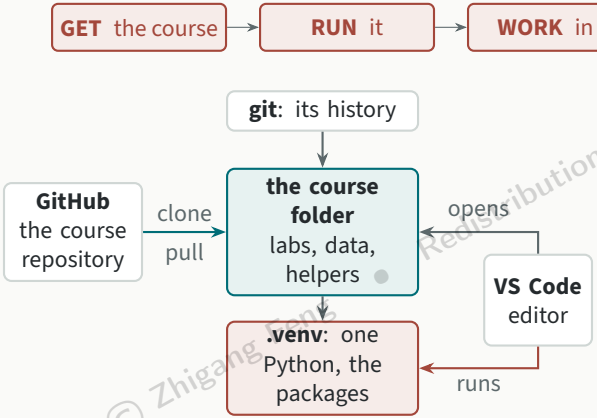
Four Messages You Will Meet

what you see	what it means	fixed in
`command not found: git`	the shell cannot find the program	GET
`ModuleNotFoundError: No module named 'torch'`	the package is not in the Python running your code	RUN, WORK
`NameError: name 'beta' is not defined`	a notebook depended on a cell it never reran	WORK
"It worked yesterday."	something changed, and nobody can say what	KEEP

Redistribution prohibited without permission
© Zhiyuan Feng

- Four parts each remove one: GET the course, RUN it, WORK in it, KEEP it. A fifth, WRITE, teaches the Python itself.
- **None of the four is a Python problem; all four are workbench problems, and each costs more hours than the economics it blocks.**

The Workbench in One Picture



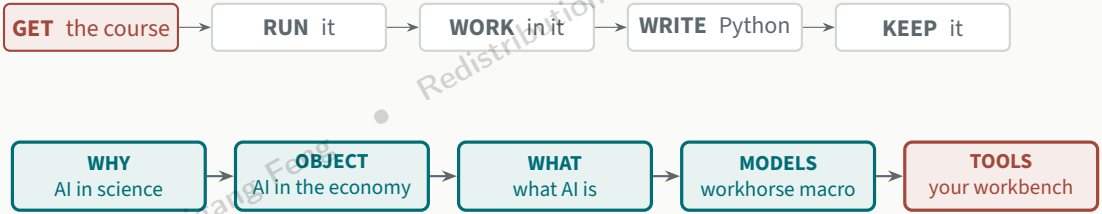
© Zhigang Feng
Redistribution prohibited without permission

- **One folder**, cloned from GitHub; **one environment** inside it; **one editor** pointed at both; **one history** watching the folder.
- The editor runs nothing itself: it hands your code to the Python it is pointed at.
- **The parts build these pieces in order; WRITE fills the folder with Python.**

Session 1.5

Part I · GET the Course

Three installs, a terminal, and one clone



© Zhigang Feng Redistribution prohibited without permission

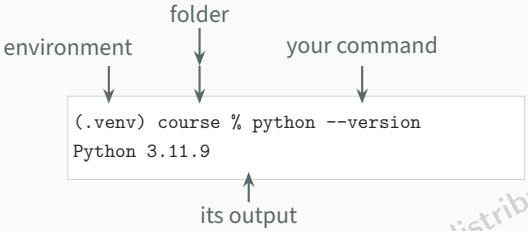
Three Installs, Two Operating Systems

	Windows	macOS
Python 3.11 the interpreter	the 3.11.9 installer from python.org; tick *Add python.exe to PATH*, off by default	the 3.11.9 installer from python.org; the command is `python3`
Git the history tool	Git for Windows, from git-scm.com	the command-line tools: `xcode-select --install`
VS Code the editor	code.visualstudio.com	code.visualstudio.com

© Zhiyang Feng Redistribution prohibited without permission

- Three separate programs: installing VS Code installs neither Python nor git. If python.org is slow, the Huawei Cloud mirror carries the same installers.
- **Three installs, done once, before Session 2; check each with `--version`.**

The Terminal: Where Commands Go

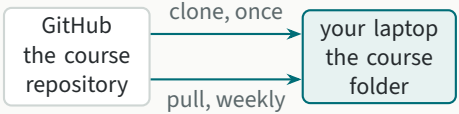


- A **shell** runs the commands you type: Terminal on macOS, PowerShell on Windows, or the terminal inside VS Code.
- `cd` enters a folder, `ls` (`dir`) lists it; `--version` asks a program which one it is.
- **PATH** is the list of folders searched for a program: `command not found` ("not recognized" on Windows) means it is on none of them.
- **Removes `command not found`: install, then open a new terminal.**

© Zhigang Feng
Redistribution prohibited without permission

Clone the Course Once, Pull Every Week

```
git clone https://github.com/vonzhg/AI-ECON-2026.git     # once
cd AI-ECON-2026
git pull                                                  # before each session
```



the lab reads helpers and data from the folders beside it

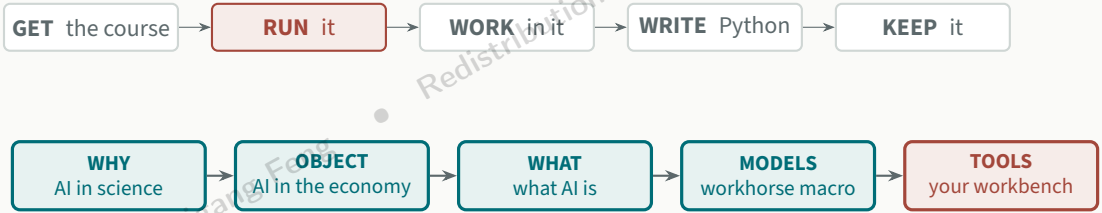
- Cloning a public repository needs no account; `git pull` brings each new lab.
- A notebook downloaded alone fails: it needs the helpers and data beside it.
- On a slow line, `git clone --depth 1` fetches only the latest version.
- Clone into `~/code`, not a OneDrive or iCloud folder (Part V).
- **The folder is the unit: clone it whole, update it with pull.**

© Zhigang Feng Redistribution prohibited without permission

Session 1.5

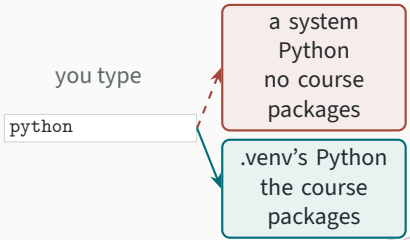
Part II · RUN It

One Python, one environment, the course's packages



Redistribution prohibited without permission
© Zhigang Feng

Which Python Is Running?



PATH is searched in order; activating `.venv` puts its Python first

- A laptop can carry several Pythons. A command runs the first one found on PATH.
- `python --version` says which version; `which python` (`where python` on Windows) says which folder it lives in. Inside the course it should be the folder's `.venv`.
- The prompt tells you too: `(.venv)` at its start means the environment is active.
- **Run both commands before asking anyone for help: every answer depends on which Python.**

© Zhigang Feng Redistribution prohibited without permission

Environments: One Project, One Python, One Set of Versions

```
python3 -m venv .venv            # macOS: once, inside the course folder
source .venv/bin/activate        #        then in every new terminal
py -3.11 -m venv .venv           # Windows: once
.venv\Scripts\Activate.ps1       #          then in every new PowerShell
```

course folder
`.venv`: torch, numpy 2

another project
its own `.venv`: numpy 1

two environments never share a version, so they never fight

PowerShell refuses the script? Once:

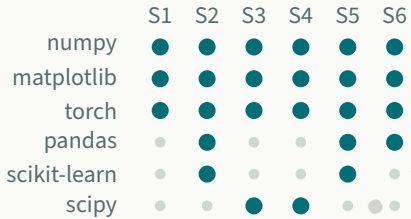
```
Set-ExecutionPolicy RemoteSigned
-Scope CurrentUser
```

- An **environment** is a folder holding one Python and one set of packages for one project.
- Install *into* the active one with `python -m pip`. Broken? Delete `.venv`, create it again.
- **Most `ModuleNotFoundError`s are a package installed into a different Python.**

Redistribution prohibited without permission
(C) Zhiheng Huang

Install the Course's Packages, Once

```
python -m pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple   # once
python -m pip install numpy pandas matplotlib scipy scikit-learn torch jupyter ipykernel
```



which Session lab imports what; jupyter and ipykernel run the notebooks

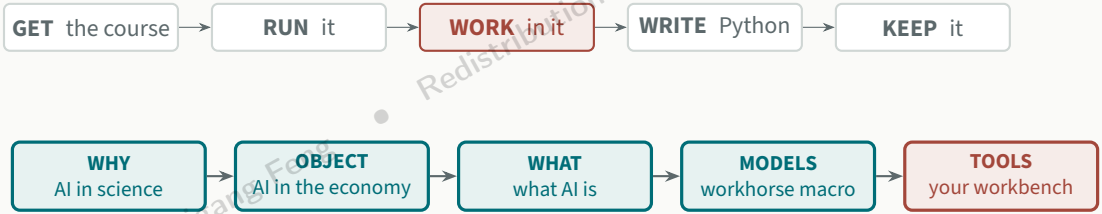
- The first line points pip at the Tsinghua mirror; the second installs all eight packages into the active `.venv`.
- The list names packages, not versions: the lab's first cell prints the versions you got. `torch` (PyTorch) is installed now and taught in Deck 2.4.
- **Removes `ModuleNotFoundError` at its source: everything the six labs import is now installed, in the Python they will use.**

© Zhigang Feng Redistribution prohibited without permission

Session 1.5

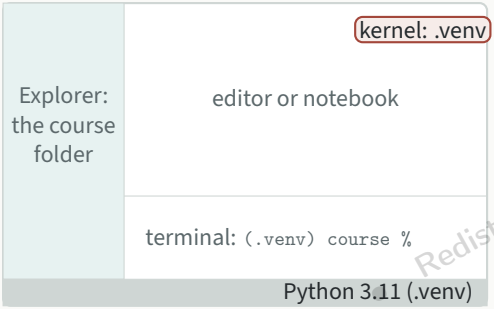
Part III · WORK in It

VS Code, the kernel, and notebooks



© Zhigang Feng Redistribution prohibited without permission

VS Code Runs Nothing Itself

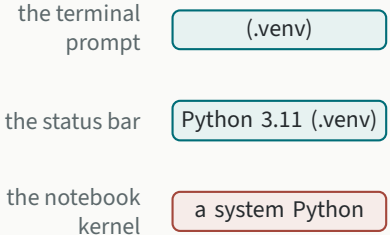


status bar: the interpreter the editor uses

- VS Code shows files and hands code to the Python you select. Install two extensions by Microsoft: *Python* and *Jupyter*.
- **File** → **Open Folder** on the course folder, not a single file; VS Code then finds its `.venv`.
- An AI assistant in the editor writes code; it does not run or check it (Deck 1.3; Session 6).
- **The editor is a window onto a folder and a Python. Get both right.**

© Zhigang Feng Redistribution prohibited without permission

Three Places Choose a Python

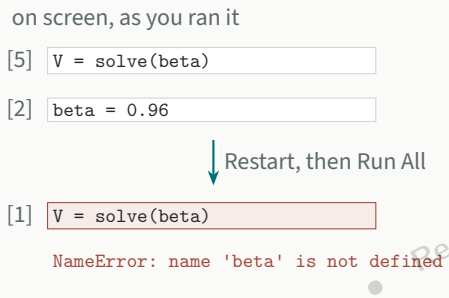


one of three on the wrong Python: the notebook cannot import torch

- All three must say `.venv`: the terminal you install in, the editor, and the notebook you run.
- **Python: Select Interpreter** sets the editor's; **Select Kernel** (a notebook's top right) sets the notebook's.
- A terminal opened inside VS Code activates the selected `.venv`; one opened elsewhere does not until you activate it.
- **Removes the other `ModuleNotFoundError`: installed, but not where the notebook looks.**

© Zhigang Feng Redistribution prohibited without permission

A Notebook Is Not a Program



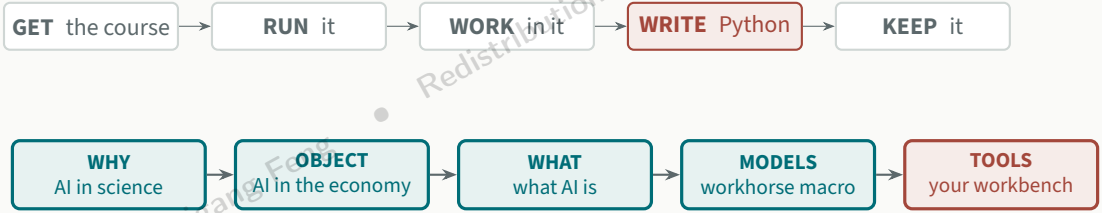
- A notebook keeps every variable until its kernel restarts, even ones whose cell you edited or deleted; the numbers in brackets record the order you ran things.
- So a notebook can look right and depend on a state nobody can rebuild.
- Before you trust or send a number: **Restart**, then **Run All**. A `NameError` then has found the hidden dependency.
- **Long or repeated runs go in a script that reads its settings at the top and writes results to disk.**

© Zhigang Feng Redistribution prohibited without permission

Session 1.5

Part IV · WRITE Python

The language the labs are written in, on the lab's growth model



© Zhigang Feng
Redistribution prohibited without permission

Values, Names and Types

```
>>> alpha = 0.33          # capital share
>>> beta = 0.96           # discount factor
>>> A = 1.0               # productivity
>>> k_ss = (alpha*beta*A) ** (1/(1-alpha))
>>> k_ss
0.17984701877776363
>>> type(alpha), type(200)
(<class 'float'>, <class 'int'>)
>>> print(f"steady state: {k_ss:.4f}")
steady state: 0.1798
```



- The lab's growth model: $k' = \alpha\beta A k^{\alpha}$, steady state $k^* = (\alpha\beta A)^{1/(1-\alpha)}$.
- `=` names a value; it is not an equation. `**` is a power.
- Types: `int`, `float`, text (`str`), `True`/`False` (`bool`); Python works them out.
- **A name points to a value; an f-string prints it.**

© Zhigang Feng Redistribution prohibited without permission

Numbers Are Not Exact

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True
>>> 1 / 2, 1 // 2, 7 // 2
(0.5, 0, 3)
```



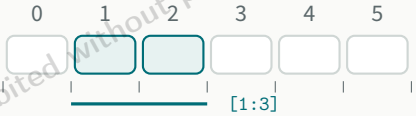
floats are a grid; 0.3 falls between two of its points

- A `float` carries about 16 significant digits, and most decimals are not exactly on its grid.
- Never test a computed float with `==`; compare with a tolerance, as `math.isclose` does.
- `//` divides and rounds down to an integer: how you turn a value into a grid position.
- **Sessions 2–4 judge a solution by residuals near 10^{-6}: know that 10^{-16} is the floor.**

© Zhaonan Feng
Redistribution prohibited without permission

Lists: A Path of Capital

```
>>> path = [0.5 * k_ss]          # start at k*/2
>>> for t in range(5):
...     path.append(alpha*beta*A*path[-1]**alpha)
...
>>> len(path)
6
>>> [round(k, 4) for k in path]
[0.0899, 0.1431, 0.1668, 0.1754, 0.1784, 0.1794]
>>> [round(k, 4) for k in path[1:3]]
[0.1431, 0.1668]
```



- A list keeps values in order; `append` adds at the end; `path[-1]` is the last.
- Positions start at 0; `[1:3]` stops *before* 3.
- `[f(x) for x in xs]` builds a list in one line. **An off-by-one index raises no error.**

© Zhigang Feng
No distribution prohibited without permission

Dictionaries and Tuples: Parameters With Names

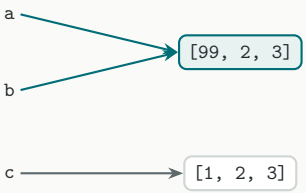
```
>>> params = dict(alpha=0.33, beta=0.96, A=1.0)
>>> params["beta"]
0.96
>>> list(params.keys())
['alpha', 'beta', 'A']
>>> s = params["alpha"] * params["beta"]
>>> y = params["A"] * k_ss ** params["alpha"]
>>> k_next, c = s * y, (1 - s) * y
>>> round(k_next, 4), round(c, 4)
(0.1798, 0.3879)
```



- A **dict** maps names to values: a calibration as one object, read by name.
- A **tuple** holds several values; `k_next, c = …` unpacks it.
- At k^* the policy returns k^* (0.1798): a check.
- **Name parameters in a dict, not in your memory.**

© Zhigang Feng
Redistribution prohibited without permission

A Name Is Not a Copy



`b = a`: a second name; `c = a.copy()`: a second object

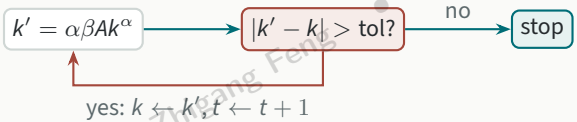
```
>>> a = [1, 2, 3]; b = a; b[0] = 99
>>> a
[99, 2, 3]
```

- Assignment never copies: `b = a` puts a second name on the same object. NumPy slices are *views* of the same memory too.
- **Value function iteration** repeats $V \leftarrow T(V)$ until the change $\max |V - V_{\text{old}}|$ is tiny.
- Write `V_old = V` and update `V` in place: both names see the update, the change is always 0, and the loop reports convergence after one step.
- **Write `V_old = V.copy()`: the bug is silent, the fix is one word.**

© Zhigang Feng
Redistribution prohibited without permission

Control Flow: Iterate to the Steady State

```
>>> k, t, tol = 0.5 * k_ss, 0, 1e-8
>>> k_next = alpha*beta*A * k**alpha
>>> while abs(k_next - k) > tol:
...     k, t = k_next, t + 1
...     k_next = alpha*beta*A * k**alpha
...
>>> print(t, round(k_next, 6), round(k_ss, 6))
15 0.179847 0.179847
```

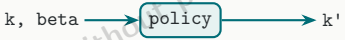


- `for t in range(5)` repeats a known number of times; `while` repeats until its test fails.
- The indented lines are the block that repeats; indentation is the syntax.
- From $k^*/2$ the loop stops after 15 steps, at k^* to six decimals.
- **Every solver in this course stops this way: on a tolerance.**

© Zhiyang Feng Redistribution prohibited without permission

Functions: Name the Rule Once

```
>>> def policy(k, alpha=0.33, beta=0.96, A=1.0):
...     "Next capital: alpha*beta*A*k**alpha."
...     return alpha * beta * A * k ** alpha
...
>>> round(policy(0.1), 4)
0.1482
>>> round(policy(0.1, beta=0.99), 4)
0.1528
>>> def step(k):
...     k_next = policy(k)
...     return k_next, k ** alpha - k_next
...
>>> [round(x, 4) for x in step(k_ss)]
[0.1798, 0.3879]
```



- `def` names a rule; `return` sends back the result; the string says what it does.
- Defaults (`beta=0.96`) change by name: `policy(0.1, beta=0.99)`.
- A function can return a tuple: k' and consumption.
- **One rule, one definition, many calls.**

© Zhigang Feng
Redistribution prohibited without permission

Modules and `import`

```
# growth.py, a file beside your notebook
ALPHA, BETA, A = 0.33, 0.96, 1.0

def policy(k):
    """Next period's capital."""
    return ALPHA * BETA * A * k ** ALPHA

if __name__ == "__main__":   # only as a script
    print(round(policy(0.1), 4))
```

```
>>> from growth import policy
>>> round(policy(0.1), 4)
0.1482
```



- A **module** is a file of Python; `import` runs it once and lends you its names. NumPy is a module: `import numpy as np`.
- The last two lines run only when the file is run as a script (`python growth.py`), not on import.
- **The labs import helpers from their folder: that is why you clone it whole.**

© Zhigang Feng
Redistribution prohibited without permission

A Model as a Class

```
class GrowthModel:
    def __init__(self, alpha=0.33, beta=0.96, A=1.0):
        self.alpha, self.beta, self.A = alpha, beta, A
        self.k_ss = (alpha*beta*A) ** (1/(1-alpha))

    def policy(self, k):
        return self.alpha*self.beta*self.A*k**self.alpha

    def simulate(self, k0, T):
        path = [k0]
        for _ in range(T):
            path.append(self.policy(path[-1]))
        return path
```

```
>>> m = GrowthModel()
>>> round(m.k_ss, 4)
0.1798
>>> path = m.simulate(0.05, 30)
>>> round(path[-1], 4)
0.1798
```

- A **class** bundles parameters (`self.alpha`) with methods.
- `__init__` runs when you make one: `GrowthModel(beta=0.99)` is another economy.
- **The lab's networks are classes like this.**

© Zhigang Feng · Redistribution prohibited without permission

Reading an Error

```
>>> policy("0.2")
Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "<console>", line 3, in policy
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'float'
>>> params["Beta"]
Traceback (most recent call last):
    ...
KeyError: 'Beta'
```

above: where, in your code

last line: what went wrong

- `NameError`: never defined. `TypeError`: wrong kind of value. `IndexError`: past the end. `KeyError`: not in the dict.
- **Read the last line first; paste the whole message when you ask for help.**

© Zhigang Feng
Redistribution prohibited without permission

NumPy: One Array, No Loop

```
>>> import numpy as np
>>> k = np.linspace(0.2*k_ss, 2*k_ss, 200)
>>> k.shape, k.dtype
((200,), dtype('float64'))
>>> k_next = alpha*beta*A * k**alpha
>>> k_next[:3].round(4)
array([0.1057, 0.1073, 0.1088])
>>> loop = [policy(x) for x in k]
>>> np.allclose(k_next, loop)
True
```

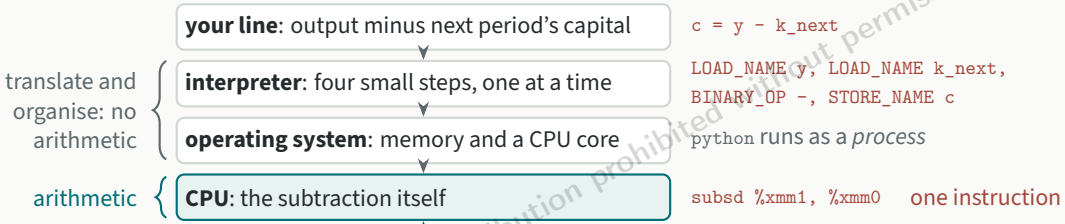


one block of 200 float64 numbers: a grid on k

- An **array** holds numbers of one type in one block of memory. `linspace` makes a grid of 200 points from $0.2k^*$ to $2k^*$.
- Arithmetic on an array applies to every element: one line computes the policy at all 200 points.
- The list loop gives the same numbers; NumPy runs its loop in compiled code, far faster.
- **On a grid, write the formula once and let the array carry it.**

© Zhigang Feng Redistribution prohibited without permission

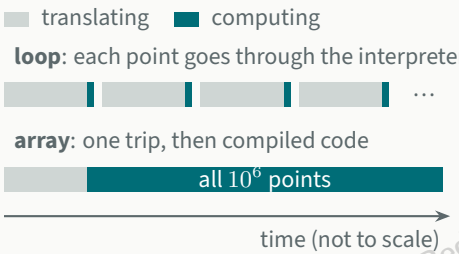
One Step Deeper: What Happens to One Line

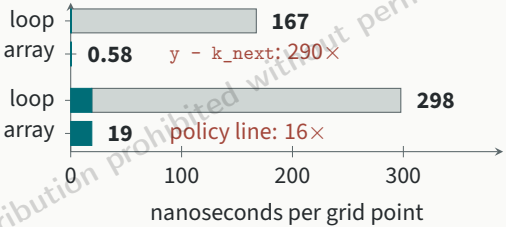


© Zijian Cheng · Distribution prohibited without permission

- The **interpreter** (`python`) is a live translator: each line becomes small steps (**bytecode**), run one by one, every time the code runs.
- **Machine code** fits one operating system and one CPU family (Intel/AMD or Apple silicon): build a `.venv` on each machine; never copy one.
- **Only the CPU computes; every layer above it translates or organises.**

Where the Time Goes: Translating Versus Computing





© Zhiyuan Chen
Redistribution prohibited without permission

- Timed on 10^6 grid points, best of 7: `c = y - k_next` takes 167 ns a point in a loop and 0.58 ns as an array; the NumPy frame's policy line, 298 against 19 ns.
- Teal is the array's time, the arithmetic itself; grey is what the loop adds. A power is real work, so there the gap is 16 times, not 290.
- Measured on one Linux machine: run `%timeit` to see yours.
- **A loop pays the translation once per point; an array pays it once per call.**

Shapes and Broadcasting

```
>>> a = np.ones(3)           # shape (3,)
>>> b = np.ones((3, 1))      # shape (3, 1)
>>> (a + b).shape
(3, 3)
>>> d = k[:, None] - k[None, :]
>>> d.shape
(200, 200)
>>> assert k_next.shape == k.shape
```



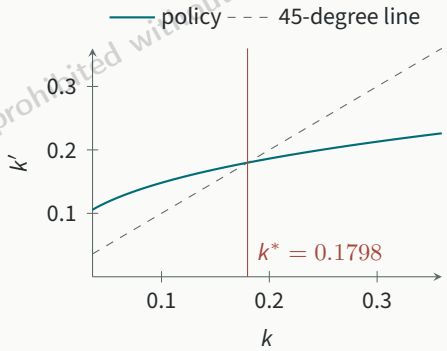
- **Broadcasting**: an axis of length 1 is stretched to match. `k[:, None] - k[None, :]` is every $k - k'$ on the grid at once, the matrix Session 2's solvers search.
- The trap: shapes (3,) and (3,1) give a (3,3) table, not three sums, and no error.
- PyTorch's tensors follow the same rules (Deck 2.4).
- **Assert the shape you expect.**

© Zhigang Feng Redistribution prohibited without permission

Plot It: The Policy Against the 45-Degree Line

```
import matplotlib.pyplot as plt
plt.plot(k, k_next, label="policy")
plt.plot(k, k, "--", label="45-degree line")
plt.axvline(k_ss, color="grey")
plt.xlabel("k"); plt.ylabel("k'")
plt.legend()
plt.savefig("policy.pdf")    # before show()
plt.show()
```

- One line per curve; `savefig` writes the file a paper uses.
- **A figure your code can redraw is a result; a screenshot is not.**

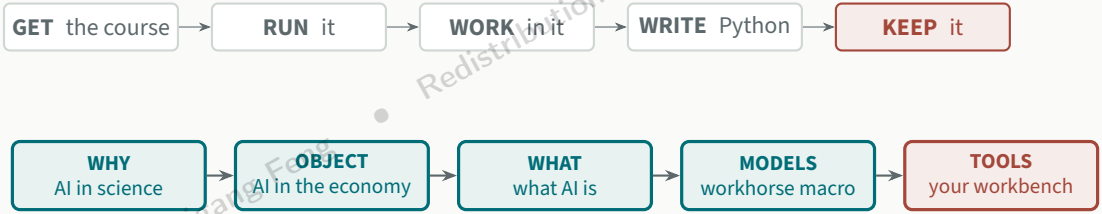


© Zejiang Feng · Redistribution prohibited without permission

Session 1.5

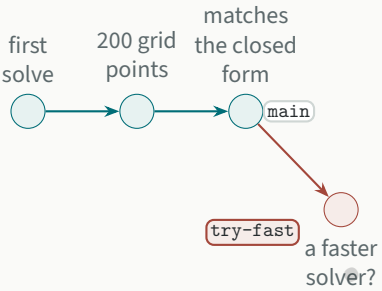
Part V · KEEP It

A history of your work, and a copy on GitHub



© Zhigang Feng
Redistribution prohibited without permission

Git: A History of Your Project



- **Git** keeps the history of a project folder. A **commit** is a snapshot of every tracked file, with a message and a link to the commit before it.
- The **history** is that chain; any commit can be compared with today or brought back. A **branch** is a line of commits you may throw away.
- A **repository** is the folder plus its history; every clone carries all of it.
- Git is not GitHub: git runs offline; GitHub keeps a copy online (Chacon & Straub, *Pro Git*, 2014).
- **Removes "it worked yesterday": every committed state can be recovered.**

© Zhigang Feng Redistribution prohibited without permission

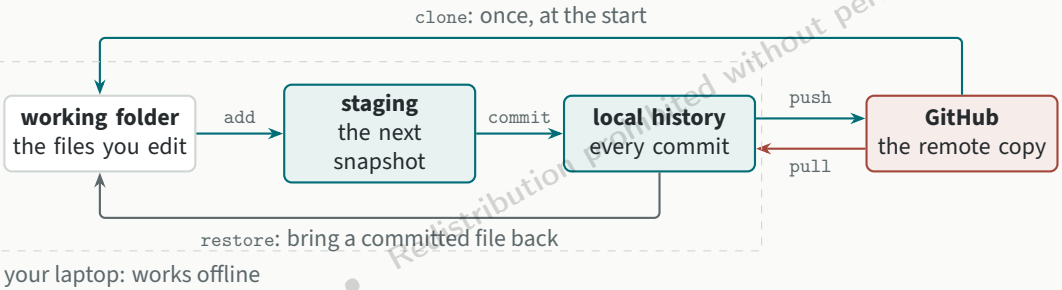
What Git Can Do for You

you want to	git does it with	where you meet it here
see what changed	`git status, git diff`	before every commit
record a result	`git add, git commit`	each time a result changes
find when it broke, go back	`git log, git diff <commit>, git restore`	“it worked yesterday”
try an idea safely	`git switch -c <branch>, git merge`	a new solver, kept apart
get the course, share your work	`git clone, git pull, git push`	weekly; your own repository
undo what an AI agent changed	`git diff, git restore`	Session 6, under an agent

© Zhiqiang Feng Redistribution prohibited without permission

- `diff` shows changes line by line; `restore` brings back a file’s last committed version; `switch -c` starts a new branch; `merge` joins it back.
- **One idea under all six rows: every commit is kept. Nothing you committed is lost, only what you never committed.**

Three Places on Your Machine, One on GitHub



© Ai-ming Feng Redistribution prohibited without permission

- `clone` copies the whole repository once. You edit in the working folder; `add` chooses the changes for the next snapshot; `commit` writes it into the history.
- `push` and `pull` exchange with GitHub: your history and GitHub's meet only then.
- **A commit is local until you push: until then it lives on one disk only.**

One File, Three Edits: Before and After You Commit

Three edits to the committed `run.py`: `N` to 400, `tol` to `1e-8`, and a typo, `beta` to `9.6`.

Save three times, commit later

```
$ git status --short
 M run.py
$ git diff
...
-beta = 0.96
-N = 200
-tol = 1e-6
+beta = 9.6
+N = 400
+tol = 1e-8
```

Git does not see saves: one tangled change; `git restore` would undo all three.

Commit after each save

```
$ git log --oneline
018f80b update beta
2d58eb9 tol 1e-8
df73dcd grid N=400
7ab99ee first VFI: 200 points
$ git revert --no-edit HEAD
[main 5cd5716] Revert "update beta"
...
```

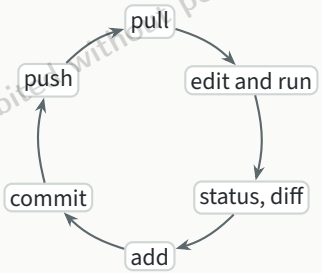
The typo sits alone in one commit; the revert restores `beta = 0.96` and keeps the two good edits.

- **A save changes the file; only a commit changes the history.**

© Zhirang Feng · Redistribution prohibited without permission

The Daily Loop

```
git config --global user.name "Your Name"   # once
git config --global user.email "you@uni.edu"
git pull          # start from the latest copy
git status        # what changed?
git diff          # how, line by line?
git add run.py
git commit -m "VFI on 200 points: max Euler error 1.2e-4"
git push          # your history -> GitHub
git log --oneline # the story so far
```



© Liyang Feng · Redistribution prohibited without permission

- Tell git who you are once; then, in your own repository, loop: pull, look at what changed, choose it, commit it with a message, push. The course folder only pulls.
- **Commit when a result changes, and put the result in the message: it is a note to yourself in three months.**

When git pull Refuses

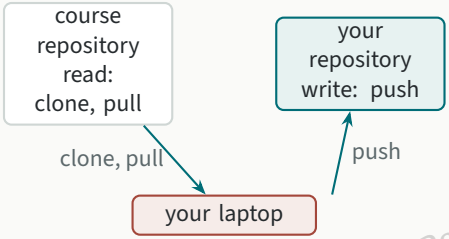
```
$ git pull
error: Your local changes to the following files would be overwritten by merge:
        lab.ipynb
Please commit your changes or stash them before you merge.
Aborting
```



- Running a notebook saves its outputs into the file, so git sees a change you never typed, and the next pull stops rather than overwrite it.
- To keep your run: save it under your own name first. To discard it: `git restore lab.ipynb`, then pull.
- **Run your own copy; let pull own the course's files.**

© Zhigang Feng Redistribution prohibited without permission

GitHub: Your Own Repository



one laptop, two remotes: one you read, one you write

- Make a GitHub account; create a repository for your own work and clone it, the verb from Part I.
- Pushing needs a sign-in, and GitHub has not accepted account passwords for git since 2021: use an SSH key, or sign in once with GitHub's `gh auth login`.
- If a laptop install fails, GitHub Codespaces runs the same notebooks in a browser.
- **Read from the course, write to your own.**

© Zhigang Feng
Redistribution prohibited without permission

What Never Goes Into a Repository

commit
code
the package list
a README: how to run it

never
data you do not own
model checkpoints
keys, passwords
`.venv`

```
# .gitignore
.venv/
*.pt
.env
```

- A key pushed once is public, and stays in the history after you delete the file.
- A `.gitignore` lists what git must never track; write it once, at the start.
- What always goes in is what someone else needs to rerun you: code, the package list, a README.
- **In Session 6 git is also the safety net under an AI agent: the habits start here.**

© Zhigang Feng
Redistribution prohibited without permission

Keep the Repository Out of OneDrive and iCloud

	a sync folder	git
what it saves	each file, alone	the whole project
when	on every change	when you commit
a message	none	one per commit
same file, two machines	keeps both, one renamed	`pull` merges, or shows the clash

one system per folder: for code, git, with GitHub as the copy

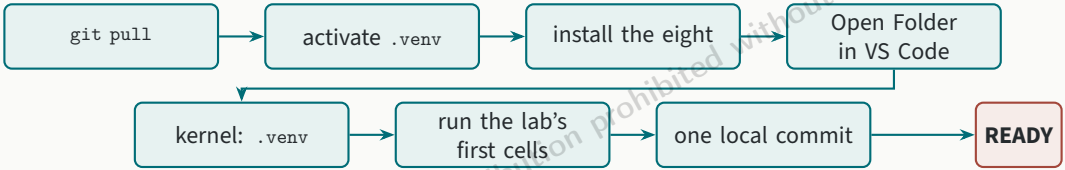
What git says when `.git/` arrives half-synced:

```
fatal: Unable to create '.../.git/index.lock': File exists.
fatal: bad object HEAD
error: bad signature 0x00000000
fatal: index file corrupt
```

- Windows often moves Documents and Desktop into OneDrive; macOS can put them in iCloud Drive.
- `.git/` is many small files that change together; a sync copies them one at a time (left).
- "Online-only" files are placeholders git cannot read.
- **Keep a repository in a plain folder such as `~/code`, outside any sync.**

© Zhigong Feng · Redistribution prohibited without permission

Ready: The Check Before Session 2



© Zhigang Feng · Redistribution prohibited without permission

- The lab's first cells print Python, every package's version and git, then find the course folder; a missing piece is named, with what to install.
- A commit of any small file in your own repository shows that git works; it needs no account.
- **If every step prints, the rest of the course runs on your laptop.**

Open the Lab, and the Bridge to Session 2

- Open the Session 1 lab notebook in the course folder, select the `.venv` kernel, and run the cells from the top. Its first cells are the check on the last slide.
- Two models, both offline on a laptop CPU with fixed seeds: a neural network that learns the growth model's saving policy and is checked against the closed form, and a small text generator trained on FOMC statements, with no benchmark inside its text.
- Fifteen minutes: the check, the two models, then three temperatures (0.5, 0.8, 1.2).
- Note privately what changed when you turned the dial: fluency, repetition, reliability.
- **Session 2 opens the first model: from one neuron to a trained network. PyTorch, NumPy's arrays plus derivatives computed for you, is Deck 2.4.**

© Zhiqiang Feng · Redistribution prohibited without permission